

Assignment 3 — Solution Key

IEEE 754 Floating-Point Representation and Arithmetic

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 8, 2026

3.1 Conceptual

Q1. A biased exponent is stored as an *unsigned* integer, so the ordering of stored exponent fields **matches the ordering of true exponent values**: the all-zeros pattern means “smallest” and the all-ones pattern “largest.” This lets the hardware compare two floats by comparing their exponent fields with a plain unsigned comparator (after the sign is checked) — no sign-extending or two’s-complement comparison is needed. That directly simplifies comparison instructions: the bit patterns themselves sort in value order. The extreme patterns are *reserved*: a stored exponent of all zeros is reserved for **zero and denormals (subnormals)**, and all ones is reserved for **infinity and NaN**.

Q2. In binary, 0.1 is a **repeating** fraction ($0.000110011\dots_2$) and is not exactly representable in 24 bits of mantissa; the hardware stores the nearest 24-bit approximation and rounds. Each accumulation therefore works on an approximate 0.1, and the ten rounding errors **accumulate** rather than cancel, so after ten additions x comes out slightly different from 1.0 — possibly on either side. The test $x = 1.0$ can therefore fail even though the exact arithmetic gives 1.0. The correct comparison is with a **tolerance**: test $|a - b| < \epsilon$ for a small application-chosen ϵ , never raw equality with `==`.

3.2 Numerical

Q3. $7.75 = 4 + 2 + 1 + 0.5 + 0.25 = 111.11_2$. Normalize: $111.11_2 = 1.1111_2 \times 2^2$. So $S = 0$; $E_{\text{true}} = 2$; biased exponent $E_{\text{stored}} = 2 + 127 = 129 = 1000\,0001_2$. Mantissa field: the fraction bits of 1.1111 , i.e. 1111 followed by zeros: $1111\,0000\,0000\,0000\,0000\,0000$. The full pattern:

$$\underbrace{0}_{\text{sign}} \quad \underbrace{1000\,0001}_{\text{exp}=129} \quad \underbrace{111100000000000000000000}_{\text{mantissa}}$$

i.e. `0x40F80000`. Check: $1.1111_2 \times 2^{129-127} = 1.1111_2 \times 2^2 = 111.11_2 = 7.75$.

Q4. `0x3F400000` = $0011\,1111\,0100\,0000\,0000\,0000\,0000\,0000$. Fields: sign = 0; exponent = $0111\,1110_2 = 126$; mantissa = $100\,0000\,0000\,0000\,0000\,0000$, i.e. fraction bits $1000\dots$. So $E_{\text{true}} = 126 - 127 = -1$, and

$$\text{value} = (-1)^0 \times 1.1_2 \times 2^{-1} = 1.5 \times 2^{-1} = 0.75.$$

Sanity check against the anchor: the exponent field is one below 127 (which is 1.0), so the value should be $1.0/2 = 0.5$ at minimum scale from the mantissa’s leading fraction bit 1.0×2^{-1} ; adding the hidden-1 makes it $1.5 \times 2^{-1} = 0.75$.

3.3 Design

Q5. *Case* $10.5 + 0.25$: $10.5 = 1.0101_2 \times 2^3$; $0.25 = 1.0_2 \times 2^{-2}$.

1. **Align:** exponent difference $= 3 - (-2) = 5$; shift 0.25 's mantissa right 5 places: $1.0_2 \times 2^{-2} \rightarrow 0.00001_2 \times 2^3$ (tracking shifted-out bits — here none are lost yet).
2. **Add:** $1.0101 + 0.00001 = 1.01011_2 \times 2^3$.
3. **Normalize:** already normalized ($1.01011_2 \times 2^3$).
4. **Round:** fits in 24 bits exactly; result $= 10.75$ exactly. No loss.

Case $1\,000\,000.0 + 0.25$: $10^6 \approx 1.11101000011001 \dots_2 \times 2^{19}$; $0.25 = 1.0 \times 2^{-2}$. **Align:** the exponent difference is $19 - (-2) = 21$ places, so 0.25 's mantissa is shifted right 21 positions — far beyond the 24-bit window of the large operand's mantissa. All of 0.25 's significant bits are shifted out and discarded. **Add / normalize / round** then leave the large mantissa unchanged. Result: the 0.25 is **lost entirely** — the sum rounds back to $1\,000\,000.0$. This is exactly the lecture's $1\,000\,000 + 0.1$ effect: the alignment step truncates the smaller operand's low bits, and the loss is worst when magnitudes differ widely.

Q6. $1.5 = 1.1_2 \times 2^0$; $2.0 = 1.0_2 \times 2^1$. Multiplication:

- **Sign:** $S = 0 \oplus 0 = 0$.
- **Exponent:** $E_{\text{true}} = 0 + 1 = 1$; stored $= 1 + 127 = 128 = 1000\,0000_2$.
- **Mantissa:** $(1.1)_2 \times (1.0)_2 = 1.1_2$; product of fractions $= 1.1$; normalized already.

So the result is $1.1_2 \times 2^{128-127} = 1.1_2 \times 2^1 = 11_2 = 3.0$ exactly. Encoding: sign 0, exponent $1000\,0000$, mantissa $1000\,0000\,0000\,0000\,0000\,0000 \Rightarrow 0x40400000$. Multiplication needs **no alignment step** because exponents just *add* — the operands' mantissas are both normalized ($1.xxx$), so multiplying them directly is valid without equalizing exponents; the only cost is that the mantissa product is up to 48 bits wide, requiring normalization and rounding.